管道&IO多路复用

管道

概述

- 命名管道就是一个比较特殊的文件，这个文件的数据存储在内存上。
- 这个管道比较特殊，无法单独打开读端或写端。必须同时打开，才能往下继续。
  如果只打开一个端，这里会阻塞住。 直到，另外一端的打开。

管道的创建

- 管道是一个比较特殊的文件，这时候我们不能使用普通的文件创建方式。
- mkfifo 1.pipe # 这个就是创建了一个管道
- echo "hello">1.pipe
  cat 1.pipe
- **千万千万不要使用vim打开管道。 如果用vim打开，这时候只能关掉tab页。这时候会卡死**

管道的使用

- 使用两个程序，一个程序打开读端，一个程序打开写端。 这时候，可以使用这个管道，从A程序给B程序发信息
- 特点1： 如果只打开读，或者写。 会卡住。
  - 比如，一个程序打开了一个pipe，使用读的方式打开。 这时候会卡住，直到有另外一个程序，使用写端的方式打开
  - pipe要手动创建。
- 特点2： 如果管道里面没有数据，会继续卡住。read会卡住
  也就是写端的数据比较慢，这时候读端会卡在read函数那里 —— **read函数调用时候会卡住**
- 特点3： 如果管道里面的数据满了。 write会卡住。
  也就是，写端比读端快
- **写满阻塞。读空阻塞。**
  前提是两端没关
- **特点4： 如果关闭1端，另外一端会怎样?**
  - 读端关了。写端继续往管道中写数据，这时候写端会收到退出的信号。然后退出程序
  - 写端关了。读端可以继续把数据读完。然后不会阻塞了。每一次读出来数据，count都为0.
- 特点5： 管道是一个半双工的
  - 一般情况下，我们使用管道，都把它当做单工用。
  - **单工、半双工、双工名词要知道**
    - 单工就是数据的单向流动的，比如收音机。
      只能信号台给收音机发消息
    - 半双工就是数据可以双向流动，但是不能同时，比如对讲机
      对讲机的一端可以给另外一端发消息，如果松掉按钮。也可以收到另外一端发的消息
    - 双工就是数据可以双向流动，而且可以同时双向，比如电话
      一个人说话的时候，另一个人也可以同时说话。

使用管道来完成两个程序的通讯

双工通讯

- 管道的打开顺序很关键。 一定要两个程序，先都打开 1.pipe，再打开2.pipe。
  否则就是死锁。
  关于命名要注意一下。
  出现了死锁，要注意怎么排查。
- 也就是，现在有两个程序，A想给B发信息。B也想给A发信息。管道我们一般当做单工用。
  这时候，我们开两个管道
- 上来先写了一个版本，这个版本就是写死的数据。
- 现在，我们想将其改成，从键盘录入信息，然后将录入的信息，发送给对方。

IO多路复用

简单说，就是系统提供的一个监听文件描述符的技术。 比如，对于标准输入来说，刚开始是处于阻塞状态，但是一旦键盘录入了数据，这时候这个文件描述符就变成了不阻塞的。
比如，对于read_fd，直接read的话，其实就是卡在这等，等到它不阻塞的时候拿出来信息。
所以多路复用，就是可以监听文件描述符，监听到了可用的文件描述符，就可以继续往下走。

select

- select函数，就是一个用于监听文件描述符状态的函数
- int select(
    int nfds, // 被监听文件描述符集合最大的文件描述符+1 (最大的文件描述符+1)
    fd_set *readfds, // 要监听的: 读操作文件描述符集合
    fd_set *writefds, // 要监听的: 写操作文件描述符集合
    fd_set *exceptfds, // 要监听的: 异常操作文件描述符集合
    struct timeval *timeout // 监听时候的阻塞时间:NULL代表一直等待直到指定就绪,0代表不等待检查文件描述符立即返回
  );
  - nfds： 被监听的文件描述符最大+1
  - readfds： 要监听的可读的文件描述符
  - timeout： 超时
- fd_set 结构体怎么构建
  - 提供一些函数，帮助我们完成构建
  - FD_ZERO: 清空监听集合
  - FD_SET: 将要监听的文件描述符，加进去
  - FD_ISSET: 判断一个文件描述符，是否在其中。是否可用
- **最后的底层流程需要注意一下**
- 平常不要为了用它而用它。 这个只是为了解决这种比较特殊的应用场景的。如果写代码中业务流程就是一来一回，其实没必要用select这种